

2212 Java Software Dev I

Program Project 4 – Ninety-Nine

Purpose

To Design and create classes, and use arrays and ArrayLists to solve a problem.

Goal

Write a program that plays a version of the card game *Ninety-Nine*.

Note: You must use the starter project and adhere to its structure when completing this assignment. Projects that are “created from scratch” will not be accepted.

Project Overview

This version of the card game *Ninety-Nine* is designed for two or more players. For every group of up to four players, use one standard pack of 52 playing cards:

- 2–4 players: 1 pack
- 5–8 players: 2 packs
- 9–12 players: 3 packs ...and so on.

Each player is dealt five cards at the start of the game. The game begins with a score of zero. Players take turns playing one card at a time. Most cards affect the total score, and the objective is to avoid playing a card that pushes the total above 99.

The effect of each card is determined by its rank:

- 2, 3, 4, 5, 6, 8 → Add the card’s rank value to the total
- 7 → Adds 7, *unless* that would exceed 99—in which case it adds just 1
- 9 → Adds 0
- 10 → Subtracts 10
- Jack → Adds 11
- Queen → Adds 12
- King → Adds 13
- Ace → Adds 14

After playing a card, a player draws a new one from the deck to maintain a five-card hand.

Another scoring rule is enforced whenever a card is played that brings the game total across any of three different thresholds in either direction. The three thresholds are 33:34, 55:56, and 77:78. When the total crosses the thresholds in either direction, 5 extra points are automatically added to the game total.

Project Specifications

To implement this game in code, begin by identifying the key classes involved. The core components are the game itself, players, cards, and the deck that holds the cards. That gives us a solid foundation.

A standard deck has 52 cards, which can be uniquely identified by a number from 0 to 51. Using simple arithmetic, we can derive the suit and rank of a card from that number:

- **Suits** (0–3): diamonds (0), clubs (1), hearts (2), spades (3)
- **Ranks** (0–12): ace (0), two (1), three (2), ..., Jack (10), Queen (11), King (12)

For example:

- Card number 0 → Ace of Diamonds
- Card number 1 → Two of Diamonds
- Card number 12 → King of Diamonds
- Card number 13 → Ace of Clubs
- Card number 14 → Two of Clubs
- Card number 25 → King of Clubs
- Card number 51 → King of Spades

The Card class should store a card's suit and rank. Its constructor receives an integer from 0 to 51 and computes the suit and rank accordingly. (It isn't necessary to keep the original number.)

The Card class should be able to return:

1. Its numeric suit and rank
2. Its standard name, e.g., *Ace of Clubs* or *Three of Hearts*.

AI Tip: Research how to solve this part using the `toString` method.

A convenient way to create the standard names is to define `String` arrays for the rank and suit names, then use the numeric values as indices to retrieve the appropriate names. Each card does NOT need its own copies of these `String` arrays, so declare them in a way that all `Card` objects share the arrays.

The Deck class is responsible for managing a collection of playing cards. Here's what it needs to do:

- Allow the creation of a deck containing one or more standard 52-card sets. The number of decks should be passed as a parameter to the constructor.
- Store the cards using an `ArrayList`, which simplifies card management.
- Shuffle the cards after they've been added to the deck. This can be done by repeatedly swapping pairs of cards at random—for example, performing 1,000 random swaps will achieve a good shuffle.

AI Tip: Explore the `Collections.swap` method to help with this.

- Provide a method to get the next card from the deck. Return `null` if the deck is empty.

AI Tip: Do some research to see if it is more efficient to remove the card from the beginning or the end of the `ArrayList`. Choose the most efficient method.

- Include a method to return the current number of cards remaining in the deck.

The Player class has a name and a collection of cards, often referred to as the player's "hand". Use an `ArrayList` to store the player's cards.

A player object needs the following behaviors:

- Retrieve the player's name.
- Get a card from the hand using a specified index.
- Play a card from the hand by removing it at a given index.
- Add a card to the hand (provided as a parameter).
- Return the current number of cards in the hand.

The `NinetyNineGame` class manages the overall gameplay. To complete the game, follow the "TODO" instructions within the `NinetyNineGame.java` file located in the starter project folder.

Program Execution Examples

When the game starts, it first displays the rules. Then, it prompts the user to enter the number of players and collects each player's name.

```
Greetings! This is the card game Ninety-Nine. Here are the rules.

Each player gets 5 cards. In a turn, each player plays a card. The point value of the card is added to
the game total, which starts at zero. The player that puts the total over 99 loses the game. After a card is
played, the player draws a new card from the deck.

Each card has a point value that is added to the total. For most of the cards, the point value is
the face value of the card. For example, the two is worth 2 points, a Jack is worth 11, a Queen is worth 12,
a King is worth 13, and an Ace is worth 14. Here are the cards with special values:
    A nine is worth 0 points
    A ten is worth -10
    A seven adds 7, unless that would put the total over 99. In that case, it adds 1.

Also, crossing the point total between 33 and 34, or 55 and 56, or 77 and 78, in either direction, adds
5 points to the total.

Have fun!

How many people are playing? 3
Player 1, enter your name: Q
Player 2, enter your name: W
Player 3, enter your name: E
```

Now that we know how many people are playing the game, we need to calculate how many packs of cards to put in the deck. *AI Tip:* Try to work out the formula on your own given the information on page 1. Then do some research and see if there is a better way.

Each player is prompted to select a card from their hand to play. Once a card is chosen, it's played, and the game total is updated accordingly. The program must also check that the selected card number is valid before proceeding.

```
There are 37 cards in the deck. The current game total is: 0

Q, it's your turn. Here is your hand:
  1) Eight of Spades  2) Six of Diamonds  3) Seven of Hearts  4) Ten of Hearts  5) Jack of Hearts

Enter the number of the card you wish to play: 0
That number is out of range. Try again: 6
That number is out of range. Try again: 5

You selected the Jack of Hearts

There are 36 cards in the deck. The current game total is: 11

W, it's your turn. Here is your hand:
  1) Five of Spades  2) Jack of Diamonds  3) Seven of Clubs  4) Queen of Hearts  5) Nine of Hearts

Enter the number of the card you wish to play: 4

You selected the Queen of Hearts

There are 35 cards in the deck. The current game total is: 23

E, it's your turn. Here is your hand:
  1) Queen of Clubs  2) Seven of Diamonds  3) Eight of Clubs  4) Ace of Diamonds  5) Five of Hearts

Enter the number of the card you wish to play: 4

You selected the Ace of Diamonds

There are 34 cards in the deck. The current game total is: 42
```

Player E played the Ace of Diamonds when the game total was 23. This added 14 points, raising the total to 37. Because the Ace caused the total to cross the 33:34 threshold, an additional 5 points were added.

```
There are 34 cards in the deck. The current game total is: 42

Q, it's your turn. Here is your hand:
  1) Eight of Spades  2) Six of Diamonds  3) Seven of Hearts  4) Ten of Hearts  5) Ten of Spades

Enter the number of the card you wish to play: 4

You selected the Ten of Hearts

There are 33 cards in the deck. The current game total is: 37
```

Next, Player Q played the Ten of Hearts. This subtracted 10 points from the game total, lowering the total to 32. Because the Ten caused the total to cross the 33:34 threshold in the other direction, an additional 5 points were added, bringing the game total to 37.

Now we will jump to the end of the game.

```
There are 27 cards in the deck. The current game total is: 98

W, it's your turn. Here is your hand:
  1) Five of Spades  2) Seven of Clubs  3) Nine of Hearts  4) Ten of Diamonds  5) Two of Diamonds

Enter the number of the card you wish to play: 2

You selected the Seven of Clubs

There are 26 cards in the deck. The current game total is: 99
```

Player W was in a tough spot but had several strong options to choose from. W played the Seven of Clubs; however, since adding 7 would have pushed the game total over 99, only 1 point was added instead.

That put player E in a tough spot.

```
There are 26 cards in the deck. The current game total is: 99

E, it's your turn. Here is your hand:
  1) Seven of Diamonds  2) Five of Hearts  3) Six of Spades  4) Two of Spades  5) Nine of Diamonds

Enter the number of the card you wish to play: 5

You selected the Nine of Diamonds

There are 25 cards in the deck. The current game total is: 99
```

Player E played the Nine of Diamonds. A 9 adds zero to the game total, thus allowing player E to survive for at least one more turn.

The game continues in this manner until a player is forced to play a card that pushes the game total past 99.

```
There are 17 cards in the deck. The current game total is: 99

E, it's your turn. Here is your hand:
  1) Seven of Diamonds  2) Five of Hearts  3) Two of Spades  4) Three of Clubs  5) King of Diamonds

Enter the number of the card you wish to play: 3

You selected the Two of Spades

GAME OVER! Sorry E, but you lost.
```

Player E had no good options. Any of the five cards would've pushed the game total past 99.

That's how the game is played!

Expectations

The program is expected to compile successfully, include identification comments at the top, and produce output that exemplifies good grammar and spelling. Failure to meet these standards may result in point deductions.